

bb-browser & OpenCLI 系统讲解

安装路径：E:\my normal txt\DigQuant\项目\SQL自适应训练\SQL自适应训练\网络爬虫

安装版本：bb-browser 0.14.2 | opencli 1.8.6

一、核心概念纠正

⚠️ **常见误解：**"bb-browser 是可视的模拟真人操作；opencli 是不可视的走背后编码"

✅ **正确理解：**两个工具的底层完全一样——都是通过 CDP (Chrome DevTools Protocol) 操控**真实的 Chrome 浏览器**。浏览器窗口里标签页切换、页面加载，你都看得见。真正的区别是**抽象层级**，不是可视与否。

二、正确的二分法

获取网页数据的方式

- 直接调 API (不可视，不需要浏览器)
 - fetch / curl → 拿 JSON
 - 例：fetch('https://gw-c.nowcoder.com/api/ ...')
- 浏览器自动化 (可视，需要 Chrome CDP)
 - bb-browser → 低层原语，灵活但繁琐
 - opencli → 高层命令，快捷但受限于适配器
 - auto-browser → 你的自研版，同理

打个比方

方式	类比
直接调 API	给仓库管理员打电话查库存 → 拿到一个数字

bb-browser (低层原语)	给你方向盘、油门、刹车，你自己开车去仓库看
opencli (高层命令)	给你一个「去仓库」按钮，按一下就到

后两种都开了真车，区别只是你握方向盘还是按自动驾驶按钮。

三、架构全景图



四、详细对比

维度	bb-browser	opencli
定位	浏览器自动化原语层	站点适配器 + CLI 生态层
抽象层级	低: click / fill / eval / snap	高: bilibili hot / zhihu hot
命令数	103 条原语命令	70-100+ 站点适配器
MCP 支持	✅ 原生 MCP Server	❌ 不直接提供
CLI Hub	❌	✅ 透传 gh / docker / tg 等
桌面应用操控	❌	✅ 可操控 Electron 应用
底层机制	CDP → Puppeteer	CDP → Puppeteer (相同)
是否可视	✅ 真实 Chrome 窗口	✅ 真实 Chrome 窗口 (相同)
GitHub Stars	~3.3k	~20k

五、bb-browser 详解

提供「原子操作」，不关心是什么网站，所有页面一视同仁：

```
# 页面操作
bb-browser open https://example.com
bb-browser snap                # 拍快照，标注所有可交互元素
bb-browser click @3            # 点第 3 号元素
bb-browser fill @5 "hello"     # 填表单
bb-browser eval "document.title" # 执行 JS
bb-browser screenshot          # 截图

# 网络抓包（比解析 DOM 高效得多）
bb-browser network start
bb-browser network requests --with-body --json
bb-browser network stop

# 站点适配器（社区贡献的高层封装）
```

```
bb-browser site zhihu/hot
```

```
bb-browser site github-repo owner=torvalds repo=linux
```

核心机制 —— Page Map:

1. `snap` 扫描页面所有可交互元素 → 生成编号快照
2. 后续用 `@N` 引用元素, 无需手写 CSS 选择器
3. 7 层检测: CSS selector → XPath → Shadow DOM → 语义匹配

六、opencli 详解

每个网站就是一个命令，一行出结构化结果：

```
# 热榜类
opencli bilibili hot -f json           # B站热门
opencli zhihu hot --limit 10          # 知乎热榜
opencli hackernews top -f table       # HN 头条

# 搜索类
opencli baidu "keyword" -n 5
opencli google "keyword"

# 开发类
opencli github trending --lang python
opencli arxiv search "LLM agent"

# CLI Hub（透传其他命令行工具）
opencli gh pr list                    # 自动检测并安装 GitHub CLI
opencli docker ps
opencli obsidian search              # 搜 Obsidian 笔记
```

opencli 独有的三大能力：

1. **CLI Hub**：不仅是浏览器工具，还是命令行聚合器，自动安装缺失工具
2. **桌面应用操控**：通过 CDP 操控 Cursor、ChatGPT Desktop 等 Electron 应用
3. **零 Token 消耗**：命令在浏览器本地执行，数据直接返回，不经过 LLM

七、选型指南

你需要什么？

- 给 AI Agent 上网能力（MCP 集成）
→ **bb-browser**（原生 MCP Server）
- 快速从常见网站提取结构化数据
→ **opencli**（高层站点适配器，一行出结果）
- 操控非标准网站、自定义交互流程
→ **bb-browser**（低层原语：**open→snap→click→fill→eval**）

- 扒 API 响应（比解析 DOM 更高效）
→ `bb-browser network capture`
- 操控桌面 Electron 应用
→ `opencli`
- 两个都要（互补）
→ 都装。`opencli` 快速出活，`bb-browser` 兜底复杂场景

八、与你已有项目的关系

	auto-browser	bb-browser	opencli
架构	CDP → Puppeteer	CDP → Puppeteer	CDP → Puppeteer
站点适配器	site/registry/*.mjs	site/ 社区适配器	内置 70+
CLI 风格	cli.mjs site nowcoder-sql	bb-browser site xxx	opencli xxx
成熟度	自用原型	3.3k ★ 社区维护	20k ★ 社区维护

你的 `nowcoder-sql.adapter.mjs` 就是标准的 `bb-browser site adapter` 写法——
`adapter` 函数签名为 `async function execute(page, params)`，返回结构化数据，社区里
叫 Tier-1 适配器。

九、总结一句话

bb-browser 和 opencli 是同一套 CDP 机制的两层皮——前者给你螺丝刀自己拧，后者给你一键开关。真正的「不可视」方案是直接调 API (fetch/curl)，那才不走浏览器。